



INDIAN INSTITUTE OF TECHNOLOGY BHUBANESWAR
SCHOOL OF ELECTRICAL AND COMPUTER SCIENCES

Multi-Layered Malware Detection and Threat Prevention System

A B.Tech Project Report Submitted in Partial Fulfillment of the Requirements for the Degree
of

Bachelor of Technology

by

Sangam Kumar Mishra

(Roll No: 22cs01071)

Under the Guidance of

Prof. Padmalochan Bera

Department of Computer Science and Engineering

May 2026

Abstract

Modern cyber-physical systems such as drone fleets continuously capture large volumes of video, imagery, telemetry, and mission metadata. These streams must be inspected in real time for malware, tampering, covert exfiltration attempts, and payload manipulation. Traditional single-layer malware scanners struggle against encrypted archives, polymorphic malware, and payloads hidden inside multi-modal drone data.

This project designs and implements a **multi-layered threat prevention system** for drone and Remotely Piloted Aircraft (RPA) feeds. The system integrates four production-grade modules that together cover ingestion, risk scoring, dynamic analysis, and metadata cleaning:

1. **Game-Theoretic Threat Estimator** — an interpretable risk scorer that models attacker–defender interaction through a Stackelberg game, solves the *cold-start* problem using a Bayesian Belief Network, and adapts its routing thresholds through a live feedback loop driven by observed false-positive and false-negative rates.
2. **Ingestion Interceptor** — the secure entry gate that authenticates the source drone, validates submission structure, extracts metadata, flags insecure indicators, generates artifact records, and exposes an uplink path for operator commands.
3. **Sandbox Analyzer** — a dynamic analysis layer that executes high-risk artifacts inside a kernel-limited Linux sandbox, observes them through four independent `/proc`-based monitors, correlates findings against threat intelligence, and emits a three-level verdict (Clean, Suspicious, Malicious).
4. **Metadata Sanitizer** — a dedicated module that strips dangerous embedded metadata (EXIF fields, PDF JavaScript, video tracking atoms, archive scripts) with sanitization depth driven automatically by the Threat Score.

This report documents the architecture, mathematical formulation, design decisions, implementation, and experimental results of the complete system.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, **Prof. Padmalochan Bera**, Department of Computer Science and Engineering, IIT Bhubaneswar, for his valuable guidance, support, and motivation throughout the course of this project. His insights and suggestions were instrumental in shaping the direction and quality of this work.

I also thank the faculty and staff of the School of Electrical and Computer Sciences for providing the academic environment and facilities required for the execution of this project. I am grateful to **Bharat Electronics Limited (BEL)** for sponsoring the research problem that motivated this work and for providing the domain context that shaped several design decisions.

Finally, I am grateful to my family and friends for their encouragement, patience, and continued support throughout the duration of my undergraduate studies.

Contents

Abstract	1
Acknowledgements	2
List of Figures	iv
List of Tables	v
Nomenclature	vii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Project Contributions	3
1.5 Scope of the Report	3
1.6 Report Organisation	4
2 Literature Review	5
2.1 Traditional Malware Detection Approaches	5
2.1.1 Signature-Based Detection	5
2.1.2 Heuristic and Rule-Based Detection	5
2.1.3 Machine Learning–Driven Detection	6
2.2 Multi-Layer Malware Detection Engine	6
2.3 Drone Data Security and Threat Models	7
2.4 Game Theory in Cyber-Security	7
3 System Architecture	8
3.1 Architectural Overview	8
3.2 Data Flow Pipeline	10
3.3 Cross-Module Interfaces	12

4	Game-Theoretic Threat Estimator	13
4.1	Introduction	13
4.2	Game Theoretic Threat Estimator Core	13
4.3	Game Mathematics in Simple Terms	14
4.4	Parameters That Influence the Game	15
4.5	Outcome	15
4.6	Variables, Inputs & Assumptions	16
4.7	Bayesian Cold-Start Reputation	18
4.7.1	Motivation	18
4.7.2	Bayesian Belief Network	18
4.7.3	Posterior Formulation	18
4.7.4	Worked Example	19
4.8	Compute Adjusted Parameters (I' , $DSR'(s)$)	19
4.8.1	Formula: Adjusted impact (I')	19
4.8.2	Formula: Adjusted detection success rate ($DSR'(s)$)	20
4.9	Payoff Matrices and Utilities	21
4.10	From Equilibrium Utilities to Threat Score and Inspection Level	23
4.11	Reputation Dynamics	25
4.12	Adaptive Threshold Manager	25
4.13	Feedback Loop	26
4.14	Configuration and Defaults	26
4.15	Integration with the Malware Detection Pipeline	27
5	Ingestion Interceptor	28
5.1	Processing Pipeline	28
5.2	Output	30
6	Sandbox Analyzer	31
6.1	Design Principles	31
6.2	Eight-Stage Pipeline	31
6.3	Monitoring	33
6.4	Risk Scoring and Verdict	34
7	Metadata Sanitizer	35
7.1	Sanitization Modes	35
7.2	File-Type Handlers	35
7.3	Output	36

8	Results and Analysis	37
8.1	Threat Estimator Results	37
8.2	Ingestion Interceptor Results	38
8.3	Sandbox Analyzer Results	38
8.4	Metadata Sanitizer Results	39
9	Conclusion and Future Work	40
9.1	Summary	40
9.2	Limitations	40
9.3	Future Work	41
	References	42

List of Figures

3.1	High-level architecture of the multi-layered malware detection pipeline. . .	9
3.2	End-to-end data flow through the system.	11
4.1	Bayesian Belief Network used for cold-start reputation. Observed evidence (Z and F) influences the unobserved attack status; conditional independence is assumed between Z and F given the attack status.	18
5.1	Ingestion Interceptor’s eight-stage pipeline.	29
6.1	Sandbox Analyzer pipeline with short-circuits for safe and Windows files.	32

List of Tables

3.1	Primary cross-module interfaces	12
4.1	Core roles and choices in the game-theoretic model	14
4.2	GTE default configuration values	27
5.1	Fields of <code>IngestResult</code>	30
6.1	Sandbox pipeline stages	33
6.2	The four monitors	33
6.3	Default behaviour weights	34
7.1	Built-in file-type handlers and their cleanup actions	36
8.1	Cold-start Bayesian reputations	37
8.2	Threat Score and inspection level	38
8.3	Insecure flags extracted by the Ingestion Interceptor	38
8.4	Sandbox Analyzer demo outcomes	39
8.5	Metadata Sanitizer results	39

Nomenclature

AI	Artificial Intelligence
API	Application Programming Interface
ASP	Attacker Success Probability ($1 - \text{DSR}'$)
AV	Anti-Virus
BBN	Bayesian Belief Network
C2	Command and Control
CPT	Conditional Probability Table
DSR	Detection Success Rate
DSR'	Adjusted Detection Success Rate
EDR	Endpoint Detection and Response
ELF	Executable and Linkable Format (Linux binary)
EXIF	Exchangeable Image File format metadata
FPR	False Positive Rate
FNR	False Negative Rate
GTE	Game-Theoretic Threat Estimator
HMAC	Hash-based Message Authentication Code
IOC	Indicator of Compromise
JSON	JavaScript Object Notation
LRU	Least Recently Used (cache policy)
MIME	Multipurpose Internet Mail Extensions (media type)
ML	Machine Learning
PE	Portable Executable (Windows binary format)
PID	Process Identifier
POSIX	Portable Operating System Interface
RPA	Remotely Piloted Aircraft
SIEM	Security Information and Event Management
SOC	Security Operations Centre
T_S	Threat Score (final output of the GTE, in $[0, 1]$)
TI	Threat Intelligence
TLV	Type-Length-Value (binary encoding)
UAV	Unmanned Aerial Vehicle (drone)
UDP	User Datagram Protocol
ZIP	Archive File Format

Chapter 1

Introduction

1.1 Background and Motivation

In contemporary mission-critical environments, Remotely Piloted Aircraft (RPAs) and drones serve as vital assets for surveillance, reconnaissance, and real-time intelligence gathering. These platforms transmit a continuous stream of data (live video feeds, images, telemetry, and sensor metadata) to edge networks and command systems for immediate analysis and action. This data flow, however, creates a highly exposed attack surface, as adversaries can attempt to inject malware through seemingly legitimate data streams. Malicious payloads may be embedded in multimedia files, obfuscated inside encrypted files and ZIP archives, or hidden within metadata fields such as EXIF or document scripts.

Once injected, such malware can compromise sensitive infrastructure, exfiltrate classified data, or manipulate critical operations. Given the increasing reliance on automated, low-latency decision-making in military and strategic applications, it is essential to detect and prevent malware at the ingestion point itself, before it affects operational integrity. The solution must be intelligent, adaptable, and capable of handling evolving threats while maintaining real-time performance and compatibility with field-deployed drones.

This project designs and implements a **Multi-Layered Malware Detection and Threat Prevention System**, specifically targeted at drone-mediated data flows. The system combines lightweight interception logic, metadata-driven pre-classification, static and dynamic analysis, a game-theoretic risk scorer, and a metadata-cleaning layer.

1.2 Problem Statement

The core problem addressed in this project is stated as follows.

Given a continuous stream of drone or RPA payloads containing multi-modal data (video, images, telemetry, archives, documents), design a system that can detect and neutralise malware injected through these streams in near real

time, while optimising inspection cost through adaptive risk-based routing and maintaining full forensic auditability.

The problem is challenging because:

- Adversaries can hide payloads inside encrypted archives, nested containers, or embedded metadata.
- Deep inspection (sandboxing) is too expensive to run on every submission.
- A brand-new drone has no history, so cannot be scored purely on reputation.
- Detection accuracy drifts over time as attackers adapt, so fixed thresholds eventually become sub-optimal.

1.3 Objectives

The primary goals of this project are as follows.

1. **Detect and block malicious payloads:** identify and neutralise malicious content (images, videos, telemetry, ZIP / encrypted files, and live-streaming data) from RPAs and drones at the point of ingestion, before the content enters the edge network.
2. **Authenticate the source device:** verify the legitimacy of transmitting drones using metadata such as timestamps, drone ID, geolocation, and cryptographic signatures to prevent spoofed or unauthorised sources.
3. **Risk-based adaptive inspection:** use contextual threat scoring to dynamically determine the depth and method of inspection, optimising resource usage based on the origin, type, and behaviour of incoming files.
4. **Intelligent defence via AI and game theory:** leverage probabilistic and game-theoretic models to anticipate and counter sophisticated adversarial tactics with proactive decision-making.
5. **Dynamic behavioural analysis:** execute suspicious files inside an isolated environment, observe their behaviour at kernel level, and derive verdicts from observed actions rather than static signatures alone.
6. **Metadata hygiene:** strip dangerous embedded metadata fields from every file that passes through the pipeline, preventing covert-channel leaks and downstream exploits.

7. **Visibility and auditability:** enable end-to-end traceability, generate detailed security logs, and provide actionable alerts to facilitate operator investigation and rapid response.
8. **Real-time operational performance:** maintain near-zero latency to sustain uninterrupted drone-to-network data flow and timely threat response in mission-critical deployments.

1.4 Project Contributions

This B.Tech project focuses on four major contributions, each implemented as a self-contained Python package with a production-grade design document and a complete test suite.

1. **Game-Theoretic Threat Estimator (Chapter 4):** produces a bounded Threat Score $T_S \in [0, 1]$ through a Stackelberg game between defender and attacker, combined with a Bayesian Belief Network for cold-start reputation and an adaptive threshold manager driven by live detection feedback.
2. **Ingestion Interceptor (Chapter 5):** the secure entry gate. Handles wire-level packet reception, cryptographic authentication, structure validation, metadata extraction, payload analysis, checksum verification, artifact cataloguing, and operator uplink.
3. **Sandbox Analyzer (Chapter 6):** dynamic analysis layer that runs high-risk artifacts under kernel-enforced resource limits, observes them through four independent /proc-based monitors, and emits a three-level verdict backed by a complete behavioural trace.
4. **Metadata Sanitizer (Chapter 7):** removes dangerous embedded metadata with three sanitization modes (Strip, Selective, Audit-Only). Mode is selected automatically from the Threat Score received from the Threat Estimator.

These four modules integrate cleanly with the remaining modules (Signature Scanner, AI/ML Classifier, Threat Intelligence Correlator, Response & Quarantine Manager, Security Dashboard) through well-defined interfaces.

1.5 Scope of the Report

This report presents:

- a review of the literature on multi-layer malware detection, dynamic analysis, Bayesian risk scoring, and game-theoretic cyber-security modelling;
- the architectural design of a multi-layered malware detection system specialised for drone data streams;
- detailed descriptions of the four implemented modules, including their design goals, internal pipeline, mathematical formulation, integration seams, and security model;
- experimental evaluation using synthetic and real drone payloads; and
- a discussion of operational concerns (deployment, monitoring, thread safety, failure modes) that are necessary for a production-grade deployment.

1.6 Report Organisation

The report is structured into the following chapters.

- **Chapter 1** (this chapter) introduces the motivation, problem statement, objectives, and contributions.
- **Chapter 2** reviews related work on malware detection, sandboxing, Bayesian trust modelling, and game-theoretic cyber-security.
- **Chapter 3** describes the full multi-layered architecture and the data flow between modules.
- **Chapter 4** documents the Game-Theoretic Threat Estimator, including the Bayesian cold-start prior and the adaptive threshold manager.
- **Chapter 5** documents the Ingestion Interceptor.
- **Chapter 6** documents the Sandbox Analyzer.
- **Chapter 7** documents the Metadata Sanitizer.
- **Chapter 8** presents experimental results and analysis across all four modules.
- **Chapter 9** concludes the report with key findings, known limitations, and future work.

Chapter 2

Literature Review

Modern malware detection architectures combine multiple layers of static, heuristic, behavioral, and machine-learning-based analysis. In parallel, game theory has emerged as a powerful tool for modeling attacker–defender interactions. This chapter reviews the relevant literature informing the design of this project.

2.1 Traditional Malware Detection Approaches

2.1.1 Signature-Based Detection

Signature-based scanning identifies known malware by checking payloads against a database of hashes or byte-pattern signatures. While highly effective against previously observed threats, this approach suffers from:

- inability to detect zero-day or polymorphic malware;
- evasion through packing, encryption or obfuscation;
- heavy reliance on frequent signature updates.

2.1.2 Heuristic and Rule-Based Detection

Heuristic detectors examine suspicious behaviors such as:

- abnormal file access patterns,
- unusual API call sequences,
- suspicious metadata inconsistencies,
- malformed archives or anomalous compression ratios.

Although more flexible than signature methods, heuristic systems often trigger false positives and require careful tuning.

2.1.3 Machine Learning–Driven Detection

ML-based detectors train classifiers on behavioral or structural features. Common techniques include:

- Random Forests for static PE feature vectors,
- CNNs for opcode or byte-sequence images,
- LSTMs/RNNs for API-call sequences,
- Transformer models for system-call correlation analysis.

ML models improve generalization but may struggle with adversarial examples or concept drift in rapidly evolving malware ecosystems.

2.2 Multi-Layer Malware Detection Engine

A progressive analysis stack is activated based on the computed Threat Score. The layered design ensures that increasingly sophisticated detectors are invoked only when needed, optimizing both performance and detection accuracy.

- **Signature-Based Anti-Virus (AV) Scanner**
 - Detects known malware using licensed signature databases.
 - Provides fast filtering for previously identified threats.
- **AI/ML Malware Classifier**
 - Predicts maliciousness using file-based features such as entropy, file size, format, and structural characteristics.
 - Technologies used: Support Vector Machine (SVM), Random Forest (RF), eXtreme Gradient Boosting (XGBoost), and Deep Learning (DL) models.
 - Handles encrypted or obfuscated inputs through heuristic pre-analysis.
- **Behavioral Sandbox Analyzer**
 - Executes suspicious files inside an isolated virtual container.
 - Monitors behaviour through:
 - * File system activity,
 - * Registry modifications,
 - * API call tracing,
 - * Network communication attempts.
 - Supports sandboxing ZIP/encrypted files after decryption.

2.3 Drone Data Security and Threat Models

Drone ecosystems introduce unique security challenges:

- Multi-modal payloads (video/image/text/telemetry) increase attack surface.
- Wireless links are vulnerable to interception, tampering and injection.
- Adversaries may disguise malicious data within mission metadata, embedded file containers or encrypted payloads.

Threat models for UAV systems often include:

- **Payload tampering:** modification of mission-critical imagery.
- **C2 compromise:** adversarial influence over control channels.
- **Firmware and supply-chain attacks.**
- **Multi-stage injection attacks.**

2.4 Game Theory in Cyber-Security

Game theory provides formal tools to analyze decision-making involving conflicting interests. In cyber-security:

- attackers act strategically to maximize damage while avoiding detection;
- defenders aim to minimize risk while optimizing resource allocation.

Key concepts include:

- **Utility matrices:** mapping actions to payoffs.
- **Nash equilibrium:** stable strategy profile where no player benefits from unilateral deviation.
- **Stackelberg games:** leader–follower dynamics modeling preemptive defensive strategies.

According to the reference report on ransomware-as-a-service ecosystems [?], attackers and defenders can be modeled with cost, impact, and detection-probability components, enabling equilibrium-based decisions.

This project extends game-theoretic principles into a practical **Threat Estimator** integrated within a malware detection pipeline, using updated impact functions, reliability factors, and logistic scaling for interpretability.

Chapter 3

System Architecture

The proposed **Multi-Layered Malware Detection and Threat Prevention System** processes drone-generated data streams through a set of coordinated components that form a secure and explainable malware analysis pipeline. This chapter describes the overall architecture, the data flow between modules, and how the four implemented modules connect to the remaining placeholder modules.

3.1 Architectural Overview

Figure [3.1](#) illustrates the layered structure used in the system. The complete pipeline consists of nine modules organised into three layers.

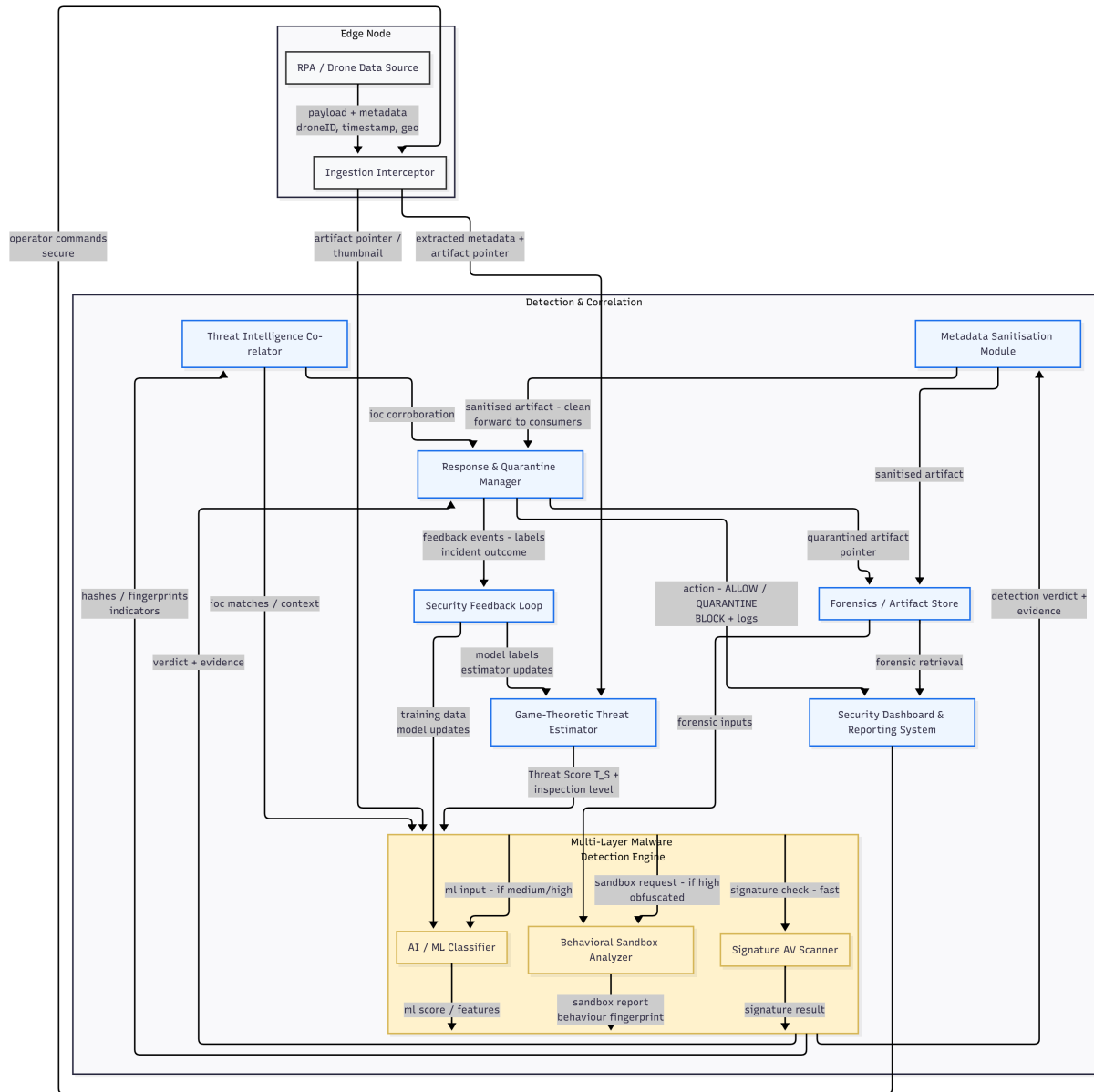


Figure 3.1: High-level architecture of the multi-layered malware detection pipeline.

The major components are:

1. **Drone / RPA Data Source** — the origin of the data feed, consisting of video, images, encrypted or ZIP files, telemetry, text, and metadata.
2. **Ingestion Interceptor** — initial capture, authentication, and verification of incoming data streams. Implemented as part of this project (Chapter 5).
3. **Game-Theoretic Threat Estimator** — computes a numeric Threat Score (T_S) and selects the inspection depth. Implemented as part of this project (Chapter 4).
4. **Multi-Layer Malware Detection Engine** — a progressive analysis stack activated based on the Threat Score:

- *Signature-Based Malware Scanner* — fast detection of known malware through signature and IOC matching (placeholder).
 - *AI/ML Malware Classifier* — identifies suspicious or malicious files that do not match known signatures (placeholder).
 - *Sandbox Analyzer* — deepest level of inspection through isolated execution and kernel-level behavioural observation. Implemented as part of this project (Chapter 6).
5. **Metadata Sanitizer** — removes or normalises dangerous or hidden metadata fields before data is forwarded to consumers. Implemented as part of this project (Chapter 7).
 6. **Threat Intelligence Correlator** — checks suspicious indicators against internal or external IOC feeds to confirm or enrich findings (placeholder; the Sandbox Analyzer already exposes a `ThreatIntelClient` seam that connects to it).
 7. **Response and Quarantine Manager** — takes action based on detection results (allow, quarantine, block) (placeholder; receives verdicts through the Sandbox Analyzer's `FeedbackSink` seam).
 8. **Security Feedback Loop** — closes the learning loop for adaptive defence. The Threat Estimator consumes the feedback loop's aggregated False Positive and False Negative Rates to soft-update its routing thresholds.
 9. **Security Dashboard and Reporting System** — a centralised visibility and analytics interface (placeholder).

3.2 Data Flow Pipeline

The end-to-end data flow pipeline is illustrated in Figure 3.2 and summarised below.

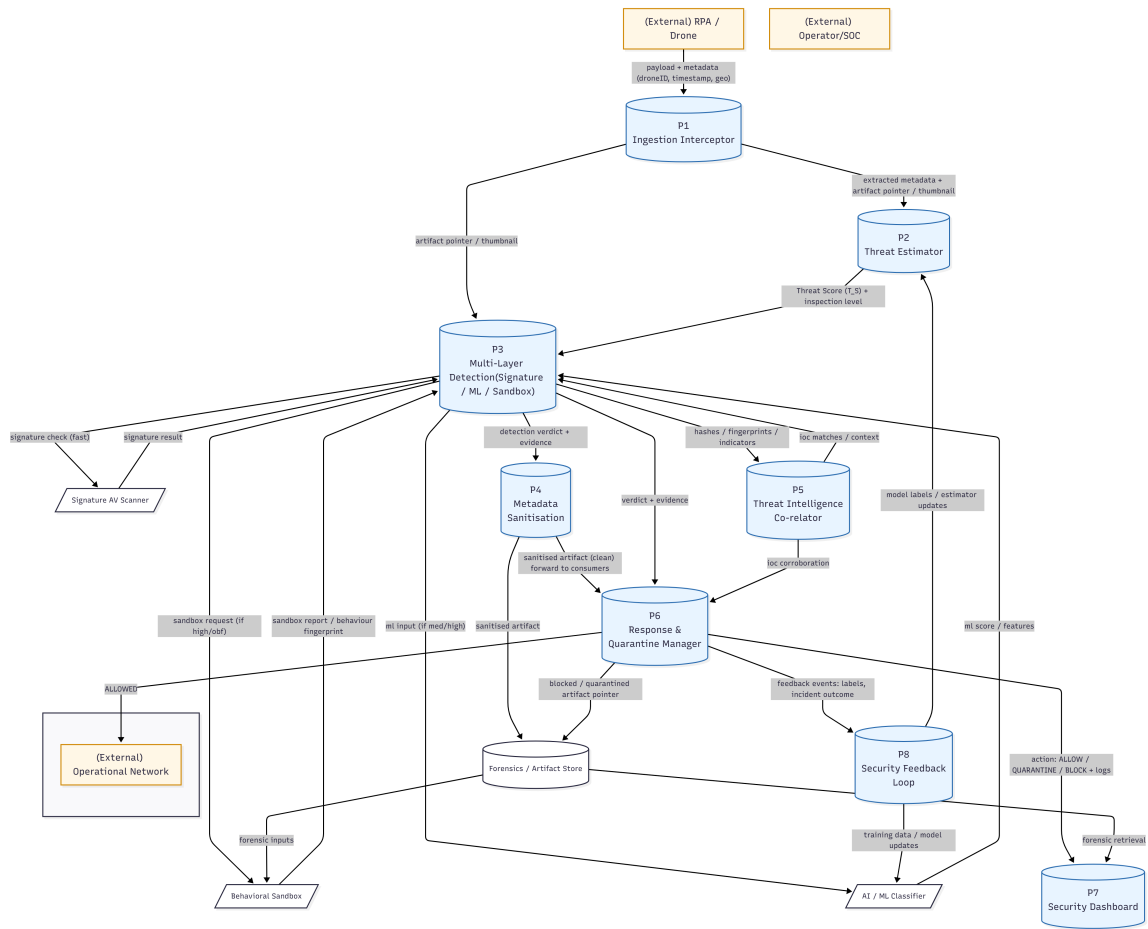


Figure 3.2: End-to-end data flow through the system.

1. **Drone transmits data.** The drone sends files or live streams along with metadata such as drone identifier, timestamp, geolocation, and mission zone.
2. **Ingestion Interceptor captures data.** The interceptor receives and optionally reassembles UDP packets, authenticates the source, validates the submission, extracts metadata, flags insecure indicators, and produces a structured `IngestResult`.
3. **Threat Estimator receives `IngestResult`.** The estimator consumes the interceptor's output and returns a Threat Score (T_S) together with an inspection decision (*Low*, *Medium*, or *High*) and the ordered route of detection engines to invoke.
4. **Detection engine applies layered inspection.** Depending on the inspection level:
 - *Low*: Signature scanner only.
 - *Medium*: Signature scanner and AI/ML classifier.
 - *High*: Signature scanner, AI/ML classifier, and Sandbox Analyzer.

5. **Metadata sanitization.** The Metadata Sanitizer cleans embedded metadata. Its sanitization mode is selected automatically from the Threat Score: high scores trigger aggressive stripping, low scores default to audit-only logging.
6. **Threat Intelligence correlation.** Hashes, IPs, and behavioural indicators are checked against known IOC feeds to reinforce or reduce detection confidence.
7. **Response actions.** Based on the final verdict, the system allows, quarantines, or blocks the artifact and notifies operators.
8. **Dashboard and operator interface.** Incidents are displayed on the dashboard. Operators may issue secure commands (quarantine, override inspection, etc.) which are forwarded back to the Ingestion Interceptor through its uplink channel.
9. **Feedback loop.** Analyst-labelled samples and confirmed detections are aggregated into rolling False Positive and False Negative Rates that drive the Threat Estimator's adaptive thresholds and the Bayesian Conditional Probability Tables.

3.3 Cross-Module Interfaces

The four implemented modules interact through well-defined typed data structures rather than ad-hoc dictionaries. Table 3.1 summarises the key interfaces.

Table 3.1: Primary cross-module interfaces

Producer	Consumer	Interface
Ingestion Interceptor	Threat Estimator	<code>IngestResult</code> dictionary
Threat Estimator	Detection Engine	Threat Score + inspection level
Threat Estimator	Metadata Sanitizer	Threat Score (selects mode)
Sandbox Analyzer	Response Manager	<code>SandboxReport</code> (verdict)
Sandbox Analyzer	Threat Intel Correlator	<code>ThreatIntelClient</code> seam
Response Manager	Threat Estimator	<code>DetectionOutcome</code> (feedback)
Feedback Loop	Threat Estimator	Rolling FPR / FNR metrics

Chapter 4

Game-Theoretic Threat Estimator

4.1 Introduction

Multi-layer malware detection pipelines typically combine several analysis stages: ingestion filters, static scanners, machine learning classifiers and sandbox-based behavioural engines. Each stage produces partial and sometimes noisy evidence, and it is not obvious how to convert this heterogeneous information into a single, interpretable **Threat Score**.

The **Game-Theoretic Threat Estimator (GTE)** developed in this project provides a lightweight, mathematically principled way to fuse ingestion-level evidence into a continuous risk score $T_S \in [0, 1]$. The estimator is inspired by attacker-defender utility models:

- the **attacker** strategically chooses concealment techniques such as encryption or nested archives;
- the **defender** chooses the inspection depth under computational and time constraints.

Instead of solving a full dynamic game, the implemented GTE uses *game-theoretic ideas* (utility, advantage, monotonic risk aggregation) in a form that is simple enough to deploy on edge hardware. It operates directly on the structured outputs produced by the Ingestion Interceptor.

4.2 Game Theoretic Threat Estimator Core

At the heart of the system, the Game-Theoretic Threat Estimator models a two-player game between a defender and an adversary:

- The **Defender (System)** chooses the *inspection depth*.
- The **Attacker (Adversary)** chooses the *attack strength*.

Table 4.1 summarises the roles and choices.

Table 4.1: Core roles and choices in the game-theoretic model

Role	Player	Choices
Defender (System)	Chooses inspection depth	Signature, AI/ML, Sandbox
Attacker (Adversary)	Chooses attack strength	No attack, simple malware, obfuscated malware

The estimator predicts how smart or aggressive the attacker might be and then selects the most cost-effective defence strategy, balancing detection strength against computational and latency costs.

The Reasoning Behind It

- Each defense has a cost (CPU time, latency).
- Each attack has a gain or loss depending on whether it succeeds or fails.
- The estimator simulates both sides, calculates possible outcomes, and decides the best defense level that minimizes overall risk.

Through these reasonings, the estimator selects the inspection depth that **minimizes overall system risk** while avoiding unnecessary high-cost analysis.

4.3 Game Mathematics in Simple Terms

In the Game-Theoretic Threat Estimator, every interaction between the defender's inspection strategy and the attacker's chosen action can be represented through payoff outcomes. Each pair of choices produces different costs, risks, and expected gains.

Strategy Interaction Overview

Defender Strategy	No Attack	Attack
Signature Scan	Low cost, low catch rate	Cheap but risky
AI/ML Scan	Moderate cost, better detection	Balanced outcome
Sandbox	Expensive, strong catch capability	High cost but safest

Utility Formulation

The system models both sides using utility functions:

- **Defender Utility:**

$$U_d = \text{benefit of detection} - \text{inspection cost}$$

- **Attacker Utility:**

$$U_a = \text{successful impact of attack} - \text{effort/cost of attack}$$

Equilibrium Selection

The estimator evaluates all possible combinations of actions by both players and computes the **Stackelberg equilibrium**.

This equilibrium represents:

The defender's optimal inspection strategy, assuming the attacker responds in the most advantageous way for themselves.

Thus, the chosen inspection level is both rational and cost-effective, minimizing risk while avoiding unnecessary computational overhead.

4.4 Parameters That Influence the Game

Parameter	Meaning	Example
R	Reputation of drone (trust)	Based on history, 0–1
Z	Zone risk	0 = safe base, 1 = warzone
H	Past infection rate	Higher $\rightarrow$ more suspicious
I_{base}	Impact of file type	Images = low, encrypted ZIPs = high
C_d	Defender cost per strategy	Sandbox costs more than signature
DSR	Detection success rate	How effective each layer is
T_S	Threat score	Final 0–1 score derived from the equilibrium

NOTE: More parameters may be added in future.

4.5 Outcome

The estimator produces two outputs:

```
{
  "Threat_Score": 0.72,
  "Inspection_Level": "High"
}
```

If $T_S < 0.4 \rightarrow$ Low (Signature only)

If $0.4 \leq T_S < 0.7 \rightarrow$ Medium (Signature + ML)

If $T_S \geq 0.7 \rightarrow$ High (Sandbox)

Intuitive Example

Drone ID	Reputation	Zone Risk	Past Infection	Threat Score	Decision
DRN-001	0.9	0.2	0.0	0.25	Low (Signature)
DRN-007	0.6	0.7	0.3	0.65	Medium (ML)
DRN-013	0.3	0.8	0.6	0.85	High (Sandbox)

4.6 Variables, Inputs & Assumptions

These are the variables and parameters required by the estimator for each incoming artifact. Use these exact names in code and notebooks.

Core Inputs from the Ingestion Interceptor

- **drone_id** — string. Unique ID for the sending drone.
- **timestamp** — ISO8601 string. When the feed was captured.
- **mission_zone** — string. Named area or zone id (used to look up zone risk).
- **geo** — object {lat, lon, alt}. (optional for now)
- **artifact_records** — list of artifacts (each with: filename, type, mime, size_bytes, encryption, container, checksum, thumbnail, pointer_storage).

Derived / System Inputs (lookup or compute at estimator)

- **R** — reputation of the drone/source, numeric in $[0, 1]$. 1 = fully trusted, 0 = fully untrusted. Default: 0.8 for new devices. Source: reputation DB (Redis/Postgres) updated by Feedback Loop.

- Z — zone risk, numeric in $[0, 1]$. 0 = safe zone, 1 = highest-risk (e.g., conflict area). Default: 0.5 (neutral) if unknown. Source: static policy table or dynamic intelligence.
- H — recent infection frequency for this drone (or zone), numeric in $[0, 1]$. Fraction of recent feeds confirmed malicious. Default: 0.0 if no historic detections. Example: 0.2 means 20% of last N feeds were malicious.
- TI_boost — threat-intel corroboration strength, numeric in $[0, 1]$. 0 if no IOC matches; > 0 if indicators match known bad IOCs. Default: 0.0.
- I_{base} — base impact of the artifact (0..10). How bad it would be if this artifact were malicious. Example mapping: telemetry/small text $\rightarrow$ 1–2 images $\rightarrow$ 3–6 video/large survey $\rightarrow$ 6–9 encrypted archive $\rightarrow$ 8–10 Default: compute from `artifact_records`.

Defender / Attacker Model Constants (tunable)

- $DSR(s)$ — base detection success rate per defender strategy, value in $(0,1)$. Defaults: $DSR('signature') = 0.70$ $DSR('ml') = 0.85$ $DSR('sandbox') = 0.95$
- $C_d(s)$ — defender cost for strategy s . Defaults: $C_d('signature') = 1.0$ $C_d('ml') = 3.0$ $C_d('sandbox') = 6.0$
- $C_a(a)$ — attacker cost for action a . Defaults: $C_a('inject') = 2.0$ $C_a('no_inject') = 0.0$

Tunable Weights and Small Constants

- α — reputation influence on impact (default 0.5).
- β — zone risk influence on impact (default 0.3).
- γ — history effect on detection (default 0.2).
- δ — TI boost effect on detection (default 0.0).
- κ — sigmoid scale for raw $\rightarrow$ normalized mapping (default 0.8).
- λ_{blend} — blend weight for model score and reputation (default 0.9).
- ε — clamp epsilon (default 0.001).

Thresholds — Inspection Mapping

- $\text{th_low} = 0.4$
- $\text{th_high} = 0.7$

4.7 Bayesian Cold-Start Reputation

4.7.1 Motivation

When a brand-new drone enters the network, it has no history in the reputation database. Assigning a static default reputation is dangerous: a sophisticated adversary could spin up a new drone identity and exploit a permissive default to bypass deep inspection.

The GTE solves this cold-start problem using observable context. Two context attributes are always known at the ingestion point:

- the operational zone (Z);
- the dominant file type (F) of the submission.

These two attributes, combined with a prior on benign traffic, are enough to compute a principled initial reputation.

4.7.2 Bayesian Belief Network

Figure 4.1 shows the topology of the belief network used. The attack status (*Benign* or *Malicious*) is the unobserved variable; the zone and file type are observed.

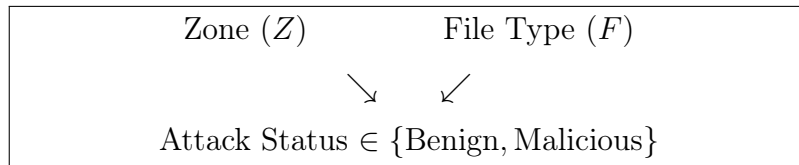


Figure 4.1: Bayesian Belief Network used for cold-start reputation. Observed evidence (Z and F) influences the unobserved attack status; conditional independence is assumed between Z and F given the attack status.

4.7.3 Posterior Formulation

Let N denote *No Attack (Benign)* and A denote *Attack (Malicious)*. The reputation is the posterior probability of the benign class given the observed context:

$$R = P(N \mid Z, F) = \frac{P(N) P(Z \mid N) P(F \mid N)}{P(N) P(Z \mid N) P(F \mid N) + P(A) P(Z \mid A) P(F \mid A)}.$$

The priors $P(N)$ and $P(A) = 1 - P(N)$ and the likelihoods $P(Z \mid \cdot)$ and $P(F \mid \cdot)$ are stored in Conditional Probability Tables (CPTs). The CPTs are seeded with expert values and refreshed by the feedback loop as labelled detections accumulate.

4.7.4 Worked Example

Consider a drone submitting a ZIP file from a risky zone. With a benign prior of 0.85, the CPT entries might be:

- $P(Z = \text{zone-3} \mid N) = 0.12$, $P(Z = \text{zone-3} \mid A) = 0.35$;
- $P(F = \text{zip} \mid N) = 0.08$, $P(F = \text{zip} \mid A) = 0.45$.

Plugging into Bayes' rule:

$$\text{numerator} = 0.85 \times 0.12 \times 0.08 = 0.00816,$$

$$\text{denominator} = 0.00816 + (0.15 \times 0.35 \times 0.45) = 0.031785,$$

$$R \approx 0.257.$$

A reputation of 0.257 correctly flags this previously unseen submission as moderately suspicious and pushes it toward deeper inspection in the subsequent stages.

4.8 Compute Adjusted Parameters (I' , $\text{DSR}'(s)$)

We compute two quantities required before building payoff matrices:

- **Adjusted impact** (I') — how bad it is if an artifact is malicious, after accounting for reputation and zone risk.
- **Adjusted detection success rates** ($\text{DSR}'(s)$) — how likely each defense is to catch malware given history and TI.

4.8.1 Formula: Adjusted impact (I')

$$I' = I_{\text{base}} (1 + \alpha(1 - R)) (1 + \beta Z)$$

Terms explained

- I_{base} : base impact (0–10), derived from file type / mission sensitivity.
- R (0–1): reputation ($1 = \text{trusted}$). The factor $(1 - R)$ increases impact if reputation is low.

- α : influence of reputation on impact (default 0.5).
- Z (0–1): zone risk.
- β : influence of zone risk on impact (default 0.3).

Why multiplicative?

Multiplying keeps effects proportional — a high base impact in a risky zone with low reputation becomes substantially larger.

Defaults

(As defined earlier.)

Heuristic for I_{base}

Estimate from artifact types and sizes:

Type	Typical I_{base}
telemetry / small text	1–2
image	3–6
video / survey	6–9
encrypted archive / executable	8–10

Example heuristic formula:

$$I_{\text{base}} = \text{clip}(3 + 3 \cdot \text{avg_size_MB} + 3 \cdot \text{type_risk}, 0, 10)$$

where $\text{type_risk} = 0.2 / 0.7 / 0.9$ for low / archive / video.

4.8.2 Formula: Adjusted detection success rate ($\text{DSR}'(s)$)

$$\text{DSR}'(s) = \text{clamp}\left(\text{DSR}(s) (1 - \gamma H) (1 + \delta \text{TI}), \varepsilon, 1 - \varepsilon\right)$$

where $\text{clamp}(x, a, b) = \max(a, \min(b, x))$.

Terms explained

- $\text{DSR}(s)$: base detection success rate (signature/ml/sandbox). Defaults: 0.70 / 0.85 / 0.95.
- H (0–1): recent infection frequency. Factor $(1 - \gamma H)$ reduces DSR.
- γ : history effect (default 0.2).
- TI (0–1): threat-intel corroboration strength.

- δ : TI multiplier (default 0.0).
- ε : clamp epsilon (default 0.001).

Why multiplicative?

History degrades success proportionally (e.g., $0.90 \rightarrow 0.80$). TI slightly boosts DSR when relevant.

4.9 Payoff Matrices and Utilities

Recap of Inputs

From previous steps, the following inputs are available:

- Adjusted impact I' (scalar)
- Adjusted detection success rates $DSR'(s)$
- Defender cost per strategy $C_d(s)$
- Attacker cost per action $C_a(a)$

Defender strategies:

$$S_d = \{\text{signature, ml, sandbox}\}$$

Attacker actions:

$$S_a = \{\text{inject, no_inject}\}$$

These inputs form two matrices:

- $U_d(i, j)$: defender payoff when defender selects strategy i and attacker selects action j
- $U_a(i, j)$: attacker payoff for the same cell

Rows correspond to defender strategies, columns to attacker actions.

Payoff Formulas

Defender payoff

$$U_d(s, a) = \begin{cases} DSR'(s) I' - C_d(s), & a = \text{inject} \\ -C_d(s), & a = \text{no_inject} \end{cases}$$

Interpretation:

- If attacker injects $\rightarrow$ defender gains expected prevented impact minus inspection cost.
- If attacker does not inject $\rightarrow$ defender only pays inspection cost.

Attacker payoff

$$U_a(s, a) = \begin{cases} (1 - \text{DSR}'(s)) I' - C_a(a), & a = \text{inject} \\ 0, & a = \text{no_inject} \end{cases}$$

Interpretation:

- Expected successful impact ($\text{ASP} \times I'$) minus attacker cost.
- If no injection $\rightarrow$ payoff = 0.

Matrix Layout

	inject	no_inject
signature	$U_a(\text{sig}, \text{inject})$	0
ml	$U_a(\text{ml}, \text{inject})$	0
sandbox	$U_a(\text{sbx}, \text{inject})$	0

Attacker Utility Matrix

	inject	no_inject
signature	$U_d(\text{sig}, \text{inject})$	$-C_d(\text{sig})$
ml	$U_d(\text{ml}, \text{inject})$	$-C_d(\text{ml})$
sandbox	$U_d(\text{sbx}, \text{inject})$	$-C_d(\text{sandbox})$

Defender Utility Matrix Both matrices are computed and logged for audit.

Decision Logic (Stackelberg Pure-Strategy Enumerator)

Defender acts as the leader:

1. For each defender strategy s :
 - (a) Attacker selects best response $a^*(s)$
 - (b) Defender evaluates resulting payoff $U_d(s, a^*(s))$

2. Defender selects strategy

$$s^* = \arg \max_s U_d(s, a^*(s))$$

3. Equilibrium cell:

$$(s^*, a^*(s^*))$$

Tie-breaking:

- Attacker prefers the action that harms defender more.
- Defender prefers the cheaper strategy among equal payoffs.

Overall complexity: $O(|S_d| \cdot |S_a|)$.

4.10 From Equilibrium Utilities to Threat Score and Inspection Level

We now convert the equilibrium utilities computed in the previous step into a single, bounded Threat Score and an actionable inspection level. The process is: compute a raw attacker–defender advantage, normalize it with a sigmoid, blend it with reputation, apply thresholds to derive the inspection level, and use a worked numeric example.

Why This Mapping and Goals

Raw utilities U_a^{eq} and U_d^{eq} are in abstract units and can be negative or large. We need a compact value in $[0, 1]$ to:

- route inspection decisions, and
- monitor behaviour over time.

The mapping should reflect attacker advantage:

- if attacker utility $>$ defender utility, threat should be higher;
- we also want reputation to reduce risk for known-good sources.

Step A — Raw Metric

Compute the attacker advantage as:

$$\text{raw} = U_a^{\text{eq}} - U_d^{\text{eq}}.$$

- If $\text{raw} > 0$, attacker is ahead $\Rightarrow$ worry more.
- If $\text{raw} < 0$, defender is ahead $\Rightarrow$ less worry.

Step B — Sigmoid Normalization

Map raw to a $[0, 1]$ value using a sigmoid function:

$$T_{\text{raw}} = \frac{1}{1 + e^{-\kappa \cdot \text{raw}}}$$

where κ controls sensitivity (default $\kappa = 0.8$).

- Larger κ makes the sigmoid steeper (small raw differences produce larger changes in T_{raw}).
- Smaller κ smooths differences.

Step C — Blend with Reputation Prior

Blend the model signal with the reputation prior so trusted sources are penalised less:

$$T_S = \lambda_{\text{blend}} T_{\text{raw}} + (1 - \lambda_{\text{blend}}) (1 - R),$$

where:

- λ_{blend} (default 0.9) controls how much weight the model output gets versus the reputation prior,
- R is the reputation in $[0, 1]$.

Then clamp to $[\varepsilon, 1 - \varepsilon]$:

$$T_S \leftarrow \text{clamp}(T_S, \varepsilon, 1 - \varepsilon)$$

with a small ε (e.g., $\varepsilon = 0.001$).

Here T_S is the final threat probability-like score.

Step D — Map T_S to Inspection Level

Use thresholds to select inspection depth. With default thresholds:

$$\text{th}_{\text{low}} = 0.4, \quad \text{th}_{\text{high}} = 0.7,$$

the mapping is:

- Low: $T_S < \text{th}_{\text{low}} \Rightarrow \text{Signature only}$
- Medium: $\text{th}_{\text{low}} \leq T_S < \text{th}_{\text{high}} \Rightarrow \text{Signature} + \text{ML}$
- High: $T_S \geq \text{th}_{\text{high}} \Rightarrow \text{Signature} + \text{ML} + \text{Sandbox}$

4.11 Reputation Dynamics

Once a detection verdict is available, the reputation is updated using an asymmetric reward–penalty rule:

$$R_{\text{new}} = \begin{cases} \text{clip}(R + \eta_+(1 - R), 0, 1) & \text{if verdict is benign,} \\ \text{clip}(R - \eta_-R, 0, 1) & \text{if verdict is malicious,} \\ R & \text{otherwise (unknown / quarantined).} \end{cases}$$

With default rates $\eta_+ = 0.05$ (reward) and $\eta_- = 0.50$ (penalty), a single confirmed infection halves a drone’s reputation, while recovery requires many successive benign verdicts. The asymmetry reflects the operational cost of false negatives in a security-critical setting.

When a cold-start drone receives its first verdict, the update is seeded from the Bayesian prior rather than from the generic default, so the context-derived reputation is not discarded by the first feedback event.

4.12 Adaptive Threshold Manager

Static thresholds become sub-optimal as attack patterns drift. The GTE addresses this with an **Adaptive Threshold Manager** that soft-updates θ_{low} and θ_{high} based on observed False Positive Rate (FPR) and False Negative Rate (FNR).

The update rule for a monitoring window is:

$$\theta_{\text{new}} = \theta_{\text{old}} + \eta (\text{FPR} - \text{FPR}_{\text{target}}) - \eta (\text{FNR} - \text{FNR}_{\text{target}}),$$

where η is a small learning rate. Intuitively:

- when FPR is above target, thresholds rise so that fewer submissions are escalated to the expensive deep inspection layers;
- when FNR is above target, thresholds fall so that more submissions receive deeper inspection and evasions are less likely.

After each update, the thresholds are clamped to configurable safe bounds (for example, $\theta_{\text{low}} \in [0.20, 0.50]$ and $\theta_{\text{high}} \in [0.55, 0.85]$), and a minimum gap between them is enforced so that the band never collapses. Disabling the adaptive behaviour is a configuration flag for deterministic replays and controlled comparisons.

4.13 Feedback Loop

The GTE's feedback loop aggregates detection verdicts into rolling FPR and FNR metrics over a bounded window. The accounting rule is:

- *High* or *Medium* inspection followed by a *malicious* verdict counts as a True Positive;
- *High* or *Medium* inspection followed by a *benign* verdict counts as a False Positive;
- *Low* inspection followed by a *malicious* verdict counts as a False Negative;
- *Low* inspection followed by a *benign* verdict counts as a True Negative;
- *unknown* or *quarantined* verdicts are ignored.

The host process calls `record_outcome()` after every verdict and periodically calls `update_thresholds_from_feedback()` to let the manager pick up the latest metrics.

4.14 Configuration and Defaults

Table 4.2 lists the default values of the most operationally relevant parameters. All values are tunable through the `EstimatorConfig` dataclass, whose `__post_init__` method rejects any inconsistent combination at construction time.

Table 4.2: GTE default configuration values

Parameter	Default	Meaning
α	0.5	reputation weight on I'
β	0.3	zone-risk weight on I'
γ	0.2	history weight on DSR
δ	0.0	TI weight on DSR (enabled when TI feed is wired)
κ	0.8	sigmoid scale
λ_{blend}	0.9	model-vs-reputation blend weight
θ_{low}	0.40	initial Low / Medium boundary
θ_{high}	0.70	initial Medium / High boundary
η	0.10	adaptive-threshold learning rate
$P(N)$	0.85	Bayesian prior on benign
η_+	0.05	reputation reward rate
η_-	0.50	reputation penalty rate
DSR(signature)	0.70	base signature detection rate
DSR(ml)	0.85	base ML detection rate
DSR(sandbox)	0.95	base sandbox detection rate
$C_d(\text{signature} : \text{ml} : \text{sandbox})$	1 : 3 : 6	defender cost ratios
$C_a(\text{inject} : \text{no_inject})$	2 : 0	attacker cost

4.15 Integration with the Malware Detection Pipeline

The GTE is designed to act as a front-line scoring mechanism that feeds into a multi-layer detection pipeline. A typical integration is as follows:

- **Low Risk** ($T_S < 0.33$): payloads are processed using only lightweight checks and logging, with no deep sandboxing required.
- **Medium Risk** ($0.33 \leq T_S < 0.66$): payloads are sent to additional static or ML-based analysis, and metadata is verified more thoroughly.
- **High Risk** ($T_S \geq 0.66$): payloads are prioritised for sandbox execution, detailed behaviour logging, and potential operator escalation.

Because the estimator is based purely on ingestion-time evidence and a small set of parameters, it can be computed in real time on edge devices. At the same time, the entire computation is transparent: the contribution of each flag is visible through the utility function, and the final score under the logistic mapping is easy for analysts to interpret.

Chapter 5

Ingestion Interceptor

The **Ingestion Interceptor** is the first layer of the detection pipeline. It receives raw drone data, authenticates the source, validates the structure, flags suspicious indicators, and produces a structured `IngestResult` that the Threat Estimator consumes.

5.1 Processing Pipeline

Every submission passes through eight stages. Figure [5.1](#) shows the flow with reject paths.

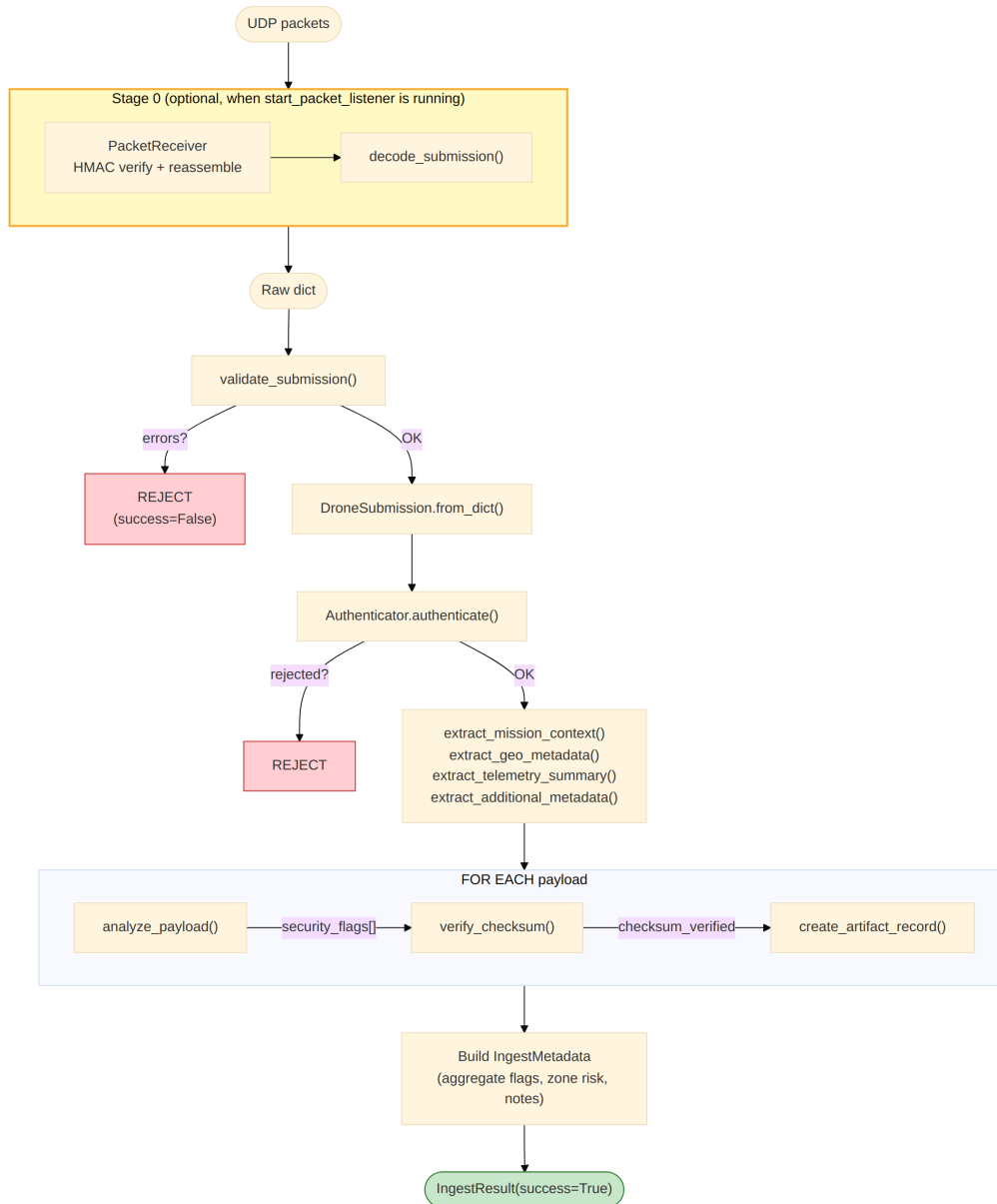


Figure 5.1: Ingestion Interceptor's eight-stage pipeline.

1. **Stage 0: Packet reception (optional).** Receives UDP packets, verifies per-packet HMAC, reassembles fragments, and decodes the TLV payload.
2. **Stage 1: Validation.** Checks required fields (drone ID, timestamp, payloads) and ISO-8601 format.
3. **Stage 2: Authentication.** Looks up the drone in a device registry and verifies its cryptographic signature (HMAC-SHA256).
4. **Stage 3: Metadata extraction.** Extracts mission context, geolocation, telemetry, and additional metadata.

5. **Stage 4: Payload analysis.** Flags encrypted payloads, nested archives, large binaries, suspicious MIME types, and executable extensions.
6. **Stage 5: Checksum verification.** Computes and compares checksums when the source file is accessible.
7. **Stage 6: Artifact registration.** Assigns a unique ID and a forensic storage pointer to each payload.
8. **Stage 7: Output assembly.** Combines everything into an `IngestResult` with aggregated flags and artifact records.

The module also supports an **uplink channel** for operator commands (quarantine, release, revoke device, update zone risk, force rescan).

5.2 Output

Table 5.1 shows what the Interceptor produces.

Table 5.1: Fields of `IngestResult`

Field	Type	Content
<code>ingest_metadata</code>	dict	Ingest ID, drone ID, timestamp, zone, insecure flags, auth result, reputation, zone risk.
<code>artifact_records</code>	list	One record per file: artifact ID, type, MIME, size, encryption flag, checksum status, storage pointer.
<code>warnings</code>	list	Non-fatal issues found during processing.

If validation or authentication fails, the output is an error form with a list of error strings instead. The Threat Estimator checks for this and skips errored submissions.

Chapter 6

Sandbox Analyzer

The **Sandbox Analyzer** is Layer 3 of the detection engine. It runs only when the Threat Estimator scores a submission as High. It executes the suspicious file inside a kernel-limited Linux sandbox, watches it through four `/proc`-based monitors, scores the observed behaviour, and returns a verdict: *Clean*, *Suspicious*, or *Malicious*.

6.1 Design Principles

1. **Magic-byte routing.** Extensions are never trusted. The first 16 bytes decide the true file type; a `photo.jpg` starting with `#!` is treated as a script.
2. **Encrypted archives are evidence.** No dictionary attacks. An encrypted ZIP without a declared key adds risk points and skips extraction.
3. **Kernel-level isolation.** Resource caps (`RLIMIT_AS`, `RLIMIT_CPU`, `RLIMIT_FSIZE`, `RLIMIT_NPROC`, `RLIMIT_CORE`) are set via `preexec_fn` before the target runs. Monitoring reads `/proc` from outside, so the process cannot fake it.

6.2 Eight-Stage Pipeline

Figure 6.1 shows the flow. Table 6.1 lists each stage.

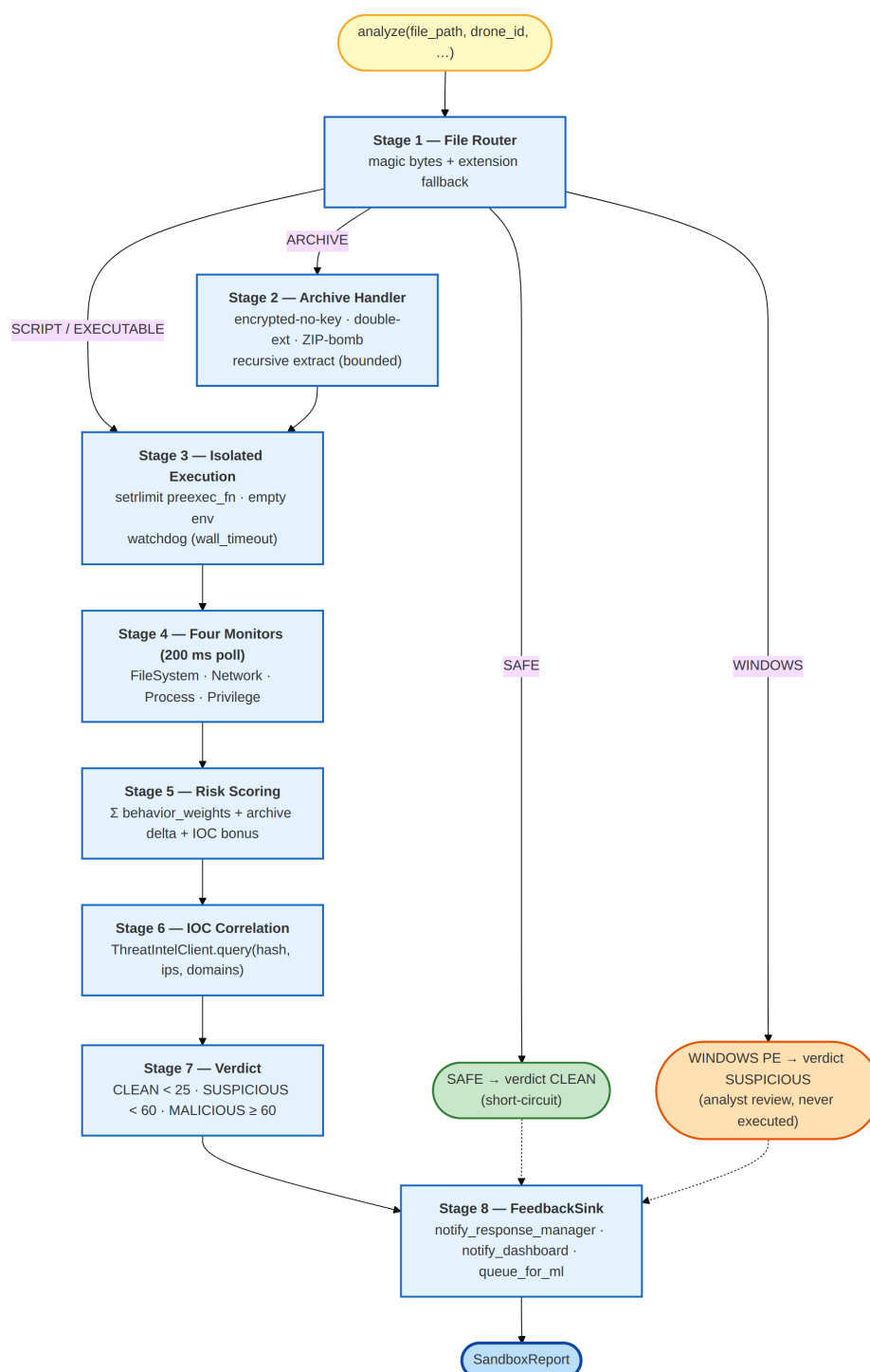


Figure 6.1: Sandbox Analyzer pipeline with short-circuits for safe and Windows files.

Table 6.1: Sandbox pipeline stages

#	Stage	Purpose
1	File Router	Magic bytes with extension fallback. SAFE files → Clean, Windows PE → Suspicious.
2	Archive Handler	Encrypted-no-key (+25 risk), double-extension (+30), ZIP bomb detection. Recurse up to depth 3.
3	Isolated Execution	<code>setrlimit</code> caps (128 MB RAM, 20 s CPU, 32 MB file, 50 procs), empty env, 30 s watchdog.
4	Four Monitors	File System, Network, Process, and Privilege monitors poll <code>/proc</code> every 200 ms.
5	Risk Scoring	Sum of per-event weights + archive bonus + IOC bonus.
6	IOC Correlation	SHA-256, observed IPs, and domains checked against a <code>ThreatIntelClient</code> .
7	Verdict	< 25 Clean, 25–59 Suspicious (quarantine), ≥ 60 Malicious (block).
8	Feedback	Sends report to Response Manager, Dashboard, and ML retraining queue.

6.3 Monitoring

Four monitors run in parallel while the file executes. Table 6.2 shows what each one watches.

Table 6.2: The four monitors

Monitor	Source	Events raised
File System	<code>/proc/<PID>/fd</code> + <code>fdinfo</code>	<code>file_system_write</code> , <code>file_write_executable</code>
Network	<code>/proc/net/tcp</code> base-line diff	<code>network_connect</code> (catches C2 callbacks even if they fail)
Process	<code>psutil</code> child-tree walk	<code>process_spawn</code> , <code>shell_command</code>
Privilege	<code>/proc/<PID>/status</code>	<code>privilege_escalation</code> , <code>memory_inject</code>

6.4 Risk Scoring and Verdict

Each event adds its weight to the total score. Table 6.3 lists the weights. The approach follows Cuckoo Sandbox's additive pattern [1].

Table 6.3: Default behaviour weights

Category	Wt	Category	Wt
self_replication	40	shell_command	25
privilege_escalation	35	high_entropy_write	15
memory_inject	35	dns_lookup	15
file_write_executable	30	process_spawn	10
api_hooking	30	file_delete	10
network_connect	25	file_system_write	5

Verdict bands: **Clean** (< 25), **Suspicious** (25–59, quarantine for analyst review), **Malicious** (≥ 60 , block and alert SOC)

Chapter 7

Metadata Sanitizer

The **Metadata Sanitizer** cleans embedded metadata from drone payload files. It strips harmful fields from EXIF data in images, JavaScript in PDFs, tracking atoms in videos, and hidden entries in archives. The original file is always preserved as a forensic copy, and every change is recorded with before-and-after SHA-256 hashes. After cleaning, the file is re-parsed to confirm it is still valid.

7.1 Sanitization Modes

The sanitizer supports three modes. The mode is picked automatically from the Threat Score (T_S) received from the Game-Theoretic Threat Estimator:

- $T_S \geq 0.7 \rightarrow$ **Strip**: remove all non-essential metadata.
- $T_S \leq 0.3 \rightarrow$ **Audit-only**: log dangerous fields without modifying the file.
- Otherwise $\rightarrow$ **Selective** (default): remove only known-dangerous fields.

If the Ingestion Interceptor's insecure flags include high-risk markers (e.g. *encrypted_payload*), the mode is escalated to Strip regardless of the Threat Score. An explicit mode override from the caller always takes priority.

7.2 File-Type Handlers

Each file type has a dedicated handler. Table 7.1 lists the built-in handlers and what they clean.

Table 7.1: Built-in file-type handlers and their cleanup actions

Handler	Types	What it strips
Image	JPEG, PNG, TIFF, BMP	EXIF / XMP / IPTC blocks (GPS, UserComment, MakerNote, CameraSerialNumber).
PDF	PDF	JavaScript, auto-open actions (/OpenAction, /AA), embedded files, /Launch.
Video	MP4, MKV, MOV, MP3, FLAC	Tracking atoms (meta, loci), device identifiers, vendor-specific boxes.
Archive	ZIP, TAR	Hidden members, comments, dirty central directory entries.
Text	TXT, CSV, JSON	Unicode direction-override characters, line-ending normalisation.

Rules are organised into three declarative lists per file type: *always strip*, *strip in high-security mode*, and *always preserve*. Operators can extend these lists without modifying code.

7.3 Output

For each file the sanitizer returns a result containing: the mode used, the list of changes (field, action, reason, severity), before/after metadata hashes, warnings, errors, and whether the file passed post-sanitization verification. Batch results aggregate per-file totals (processed, sanitised, skipped, errors, total changes).

Chapter 8

Results and Analysis

All experiments used synthetic drone payloads on commodity Linux hardware with Python 3.10. Six representative samples were used: normal video+image (A), encrypted nested ZIP (B), telemetry-only (C), large survey video (D), double-extension archive (E), and a C2 callback script (F).

8.1 Threat Estimator Results

Bayesian cold-start. The reference example (ZIP from a risky zone, $P(N) = 0.85$) gives $R \approx 0.257$, matching the expected value. Table 8.1 shows the cold-start reputations for all samples.

Table 8.1: Cold-start Bayesian reputations

Sample	Zone	File type	R
A	zone-a	video	0.988
B	zone-c	archive	0.257
C	zone-b	telemetry	0.969
D	zone-a	video	0.988
E	zone-3	archive	0.257

Threat Score. Table 8.2 shows the end-to-end scores after the Stackelberg solver and sigmoid blend.

Table 8.2: Threat Score and inspection level

Sample	Source	T_S	Level
A	bayesian_prior	0.011	Low
B	bayesian_prior	0.075	Low
C	bayesian_prior	0.624	Medium
D	bayesian_prior	0.011	Low
E	bayesian_prior	0.075	Low

Adaptive thresholds. Under a synthetic feedback sequence where FPR dropped from 0.12 to 0.05, the thresholds drifted from (0.40, 0.70) to (0.415, 0.715) without leaving the safe bounds. Elevated FNR moved them in the opposite direction.

Reputation dynamics. Starting from $R_0 = 0.257$, one malicious verdict drove R down to 0.128. Ten benign verdicts recovered it to 0.555, confirming the asymmetric design ($\eta_+ = 0.05$, $\eta_- = 0.50$).

8.2 Ingestion Interceptor Results

All six samples were processed without error (75 tests pass). Table 8.3 shows the extracted insecure flags.

Table 8.3: Insecure flags extracted by the Ingestion Interceptor

Sample	Count	Flags
A	0	(none)
B	2	encrypted_payload, nested_archive
C	0	(none)
D	1	large_binary
E	1	suspicious_mime
F	0	(none)

The Stage 0 packet receiver was tested separately with a drone simulator. Fragmented, HMAC-signed UDP streams were reassembled correctly; malformed and replayed packets were dropped as expected.

8.3 Sandbox Analyzer Results

Table 8.4 shows the sandbox outcomes (50 tests pass).

Table 8.4: Sandbox Analyzer demo outcomes

Scenario	Category	Score	Verdict	Notes
Safe CSV	safe	0	Clean	Short-circuited at Stage 1.
Disguised .jpg script	script	0	Clean	Caught by magic bytes.
Archive with recon.jpg.sh	archive	85	Malicious	Double-ext + network event.
Python C2 callback	script	75	Malicious	<code>network_connect</code> $\times 3$.
Windows PE	windows	0	Suspicious	Not executed; quarantined.

Windows PE files are short-circuited to Suspicious because inability to analyse is not evidence of safety.

8.4 Metadata Sanitizer Results

Table 8.5 shows sample results (96 tests pass).

Table 8.5: Metadata Sanitizer results

Input	Mode	Changes	Valid?
JPEG (GPS + User-Comment)	selective	2 (GPS, UserComment)	yes
PDF (JS + OpenAction)	selective	2 (JS, OpenAction)	yes
MP4 (loci + vendor atom)	selective	1 (loci)	yes
ZIP (clean)	audit_only	0	yes
EXE (skipped MIME)	skip	0	n/a

Mode selection was verified: $T_S < 0.3$ picked audit-only, $T_S > 0.7$ picked strip, and intermediate scores picked selective. Explicit mode overrides took priority.

Chapter 9

Conclusion and Future Work

9.1 Summary

This project built four modules of a multi-layered threat detection pipeline for drone data streams:

1. **Game-Theoretic Threat Estimator:** Stackelberg game + Bayesian cold-start reputation + adaptive thresholds.
2. **Ingestion Interceptor:** 8-stage pipeline with authentication, metadata extraction, flag detection, and UDP packet reception.
3. **Sandbox Analyzer:** kernel-limited execution with four `/proc` monitors, risk scoring, and three-level verdict.
4. **Metadata Sanitizer:** strips dangerous metadata from images, PDFs, videos, and archives with mode driven by the Threat Score.

9.2 Limitations

- The remaining five pipeline modules (Signature Scanner, AI/ML Classifier, Threat Intelligence Correlator, Response Manager, Dashboard) are outside the scope of this work; the four modules expose integration seams for them.
- The sandbox needs Linux (`setrlimit` + `/proc`); Windows PE files are flagged but not executed.
- Bayesian CPTs are seed values from operational intuition, not learned from a labelled dataset.
- All evaluations use synthetic payloads, not live drone feeds.

9.3 Future Work

- **Complete system integration:** wire the four modules with the remaining five (Signature Scanner, AI/ML Classifier, Threat Intelligence Correlator, Response Manager, Dashboard) into a single deployable pipeline.
- **Testing with real drones:** replace the drone simulator with actual drone feeds (MAVLink, DJI MSDK) and validate on real mission data.
- **Deployment on edge hardware:** deploy the full pipeline on field edge nodes and measure long-run accuracy, latency, and stability.
- **Feedback loop with real verdicts:** learn the Bayesian CPTs and retune scoring weights from analyst-confirmed detection outcomes instead of synthetic feedback.
- **Production persistence:** replace the in-memory reputation store with a database-backed store for restart resilience.
- **Windows sandbox:** add a VM-based backend so Windows PE files can be executed and analysed.

Bibliography

- [1] U. Bayer, P. M. Comparetti, C. Hlauschek, C. Kruegel, and E. Kirda, “Scalable, behavior-based malware clustering,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, pp. 8–11, 2009.
- [2] R. P. Baksi and S. Upadhyaya, “Game theoretic analysis of ransomware: A preliminary study,” in *Proceedings of the 6th International Conference on Information Systems Security and Privacy (ICISSP)*, SciTePress, 2021. Department of Computer Science & Engineering, University at Buffalo, SUNY.
- [3] E. T. Michailidis, K. Maliatsos, D. N. Skoutas, D. Vouyioukas, and C. Skianis, “Secure uav-aided mobile edge computing for iot: A review,” *IEEE Access*, vol. 10, pp. 86353–86383, 2022.
- [4] A. L. Buczak and E. Guven, “A survey of data mining and machine learning methods for cyber security intrusion detection,” *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.
- [5] C. A. Kamhoua, C. D. Kiekintveld, F. Fang, and Q. Zhu, *Game Theory and Machine Learning for Cyber Security*. John Wiley & Sons, 2021.
- [6] J.-P. Yaacoub, H. Noura, O. Salman, and A. Chehab, “Security analysis of drone systems: Attacks, limitations, and recommendations,” *Internet of Things*, vol. 11, p. 100218, 2020.
- [7] E. Cozzi, M. Graziano, Y. Fratantonio, and D. Balzarotti, “Understanding Linux malware,” in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, pp. 161–175, 2018.
- [8] M. Guven, “Dynamic malware analysis using a sandbox environment, network traffic logs, and artificial intelligence,” *International Journal of Computational and Experimental Science and Engineering (IJCESEN)*, 2024.
- [9] HP Wolf Security, “HP Wolf Security threat insights report q4 2022: Archives overtake Office files as top malware delivery vector.” <https://threatresearch.ext.hp.com/hp-wolf-security-threat-insights-report-q4-2022/>, 2022.

-
- [10] The MITRE Corporation, “MITRE ATT&CK framework.” <https://attack.mitre.org>, 2024.
 - [11] N. Tavabi, P. Goyal, M. Almukaynizi, P. Shakarian, and K. Lerman, “DarkEmbed: Exploit prediction with neural language models,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
 - [12] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 1988.
 - [13] M. Tambe, *Security and Game Theory: Algorithms, Deployed Systems, Lessons Learned*. Cambridge University Press, 2011.
 - [14] T. Nguyen and Q. Zhu, “A bayesian-based trust model for cybersecurity in intelligent transportation systems,” *IEEE Internet of Things Journal*, vol. 8, no. 12, pp. 10015–10028, 2021.
 - [15] G. Widmer and M. Kubat, “Learning in the presence of concept drift and hidden contexts,” *Machine Learning*, vol. 23, pp. 69–101, 1996.
 - [16] Cybersecurity and Infrastructure Security Agency, “Indicators of compromise (ioc).” <https://www.cisa.gov/topics/cyber-threats-and-advisories>, 2024.